My expression grammar is as follows:

expression → literal
| unary
| binary
| group

literal → INT | FLOAT | STRING | "true" | "false" | "null"

grouping → "(" expression ")"

unary → ("-" | "!") expression

binary → expression operator expression

operator → "==" | "!=" | "<" | "<=" | ">" | ">="
| "+" | "-" | "*" | "/"

literal holds all tokens with type literal, as well as keywords 'true' and 'false'
since they represent literal boolean vals, and null represent the literal nothing.

Precedence from low to high, Associativity (left or right)
Equality, Left
Comparison, Left
Term, Left
Factor, Left
Unary, Right

No design changes were made to the grammar.
Some changes were made to simplify code and make it more readable.

Dependencies and Setup:
No dependencies outside of the standard library.
The scanner and parser classes are in the src folder.
To run the program, run the pyrite.py file alone with a file
python src/pyrite.py
or
python src/pyrite.py [file]
Test files can be ran using:
python test/lab3/parser_error_test.py

Tests:
There are 39 tests in total for my most complex test yet

4 tests for unmatched parens:
  test_unclosed_open_paren:
    "(1 + 2", ParserError, expected "Unexpected character '@' at line 1, column 2"
  test_extra_close_paren:
    "1 + 2)", ParserError, expected "Unexpected character ')' at line 1, column 3"
  test_nested_unclosed:
    "((1 + 2)", ParserError, expected "Unexpected character '(' at line 1, column 2"
  test_empty_parens:
    "()", ParserError, expected "Unexpected character ')' at line 1, column 2"

7 tests for missing operands:
  test_missing_right_operand_plus:
    "1 +", ParserError, expected "Unexpected character '+' at line 1, column 3"
  test_missing_right_operand_minus:
    "1 -", ParserError, expected "Unexpected character '-' at line 1, column 3"
  test_missing_right_operand_star:
    "1 *", ParserError, expected "Unexpected character '*' at line 1, column 3"
  test_missing_right_operand_slash:
    "1 /", ParserError, expected "Unexpected character '/' at line 1, column 3"
  test_missing_right_operand_equality:
    "1 ==", ParserError, expected "Unexpected character '=' at line 1, column 3"
  test_missing_right_operand_comparison:
    "1 >=", ParserError, expected "Unexpected character '=' at line 1, column 3"
  test_missing_left_operand:
    "* 2", ParserError, expected "Unexpected character '*' at line 1, column 1"

4 tests for unexpected tokens:
  test_double_operator:
    "1 ++ 2", ParserError, expected "Unexpected character '+' at line 1, column 3"
  test_operator_only:
    "+", ParserError, expected "Unexpected character '+' at line 1, column 1"
  test_consecutive_literals:
    "1 2", ParserError, expected "Unexpected character '2' at line 1, column 2"
  test_bare_equality:
    "== 1", ParserError, expected "Unexpected character '=' at line 1, column 1"

4 tests for syntax errors:
  test_empty_input:
    "", ParserError, expected "Unexpected character EOF at line 1, column 1"
  test_unary_without_operand:
    "!", ParserError, expected "Unexpected character '!' at line 1, column 1"
  test_unary_minus_without_operand:
    "-", ParserError, expected "Unexpected character '-' at line 1, column 1"

```
    test_deeply_nested_unclosed:
        "((((1 + 2)", ParserError, expected "Unexpected character '(' at line 1, column 2"

6 tests for literal parsing:
    test_int:
        "42", Literal(42)
    test_float:
        "3.5", Literal(3.5)
    test_string:
        '"hi"', Literal("hi")
    test_true:
        "true", Literal(True)
    test_false:
        "false", Literal(False)
    test_nil:
        "nil", Literal(None)

4 tests for unary parsing:
    test_negate:
        "-5", Unary(Punctuator('-', 1), Literal(5))
    test_not:
        "!true", Unary(Punctuator('!', 1), Literal(True))
    test_double_not:
        "!!false", Unary(Punctuator('!', 1), Unary(Punctuator('!', 1), Literal(False)))
    test_negate_grouping:
        "-(1 + 2)", Unary(Punctuator('-', 1), Grouping(Binary(Literal(1), Punctuator('+', 1),
Literal(2))))

4 tests for binary parsing:
    test_addition:
        "1 + 2", Binary(Literal(1), Punctuator('+', 1), Literal(2))
    test_division:
        "3.5 / 2", Binary(Literal(3.5), Punctuator('/', 1), Literal(2))
    test_comparison:
        "1 >= 2", Binary(Literal(1), Punctuator('>=', 1), Literal(2))
    test_inequality:
        "false != true", Binary(Literal(False), Punctuator('!=', 1), Literal(True))

6 tests for precedence parsing:
    test_factor_before_term:
        "1 + 2 * 3", Binary(Binary(Literal(1), Punctuator('+', 1), Literal(2)), Punctuator('*', 1),
Literal(3))
    test_grouping_overrides_precedence:
```

"(1 + 2) * 3", Binary(Grouping(Binary(Literal(1), Punctuator('+', 1), Literal(2))), Punctuator('*', 1), Literal(3))

test_left_associative:

"1 - 2 - 3", Binary(Binary(Binary(Literal(1), Punctuator('-', 1), Literal(2)), Punctuator('-', 1), Literal(3)), Punctuator('+', 1), Literal(0))

test_comparison_before_equality:

"1 < 2 == true", Binary(Binary(Literal(1), Punctuator('<', 1), Literal(2)), Punctuator('==', 1), Literal(True))

test_unary_before_factor:

"-2 * 3", Binary(Unary(Punctuator('-', 1), Literal(2)), Punctuator('*', 1), Literal(3))

test_mixed:

"-1 + 2 * (3 - 4) > 0", Binary(Binary(Unary(Punctuator('-', 1), Literal(1)), Punctuator('+', 1), Literal(2)), Punctuator('*', 1), Grouping(Binary(Literal(3), Punctuator('-', 1), Literal(4))))

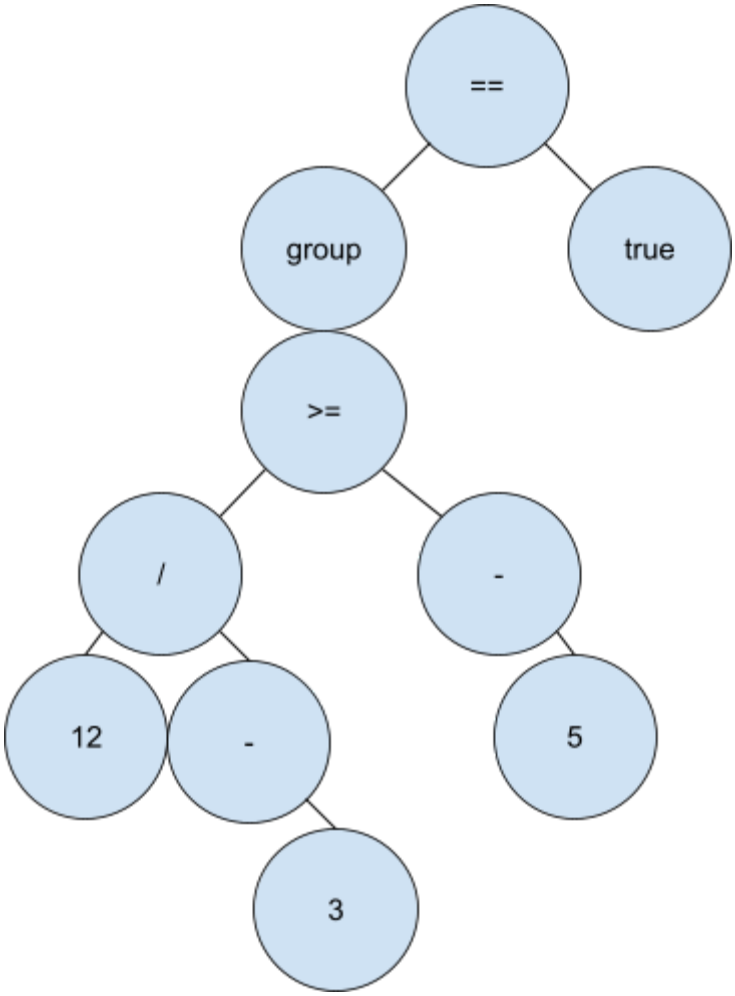
```
PS C:\Users\brody\343_Pyrite> & c:/Users/brody/343_Pyrite/.venv/Scripts/python.exe c:/Users/brody/343_Pyrite/test/lab3/parser_test.py
.....................................
----------------------------------------------------------------------
Ran 39 tests in 0.008s

OK
```

All tests pass (as well as previous tests)

Errors that can appear are as follows:
UnmatchedParenthesis which is caused by a missing ‘(‘ or ‘)’
MissingOperand when we expect another value for an operator, but aren’t given one
UnexpectedToken when we we get a token different than what we were looking for
And SyntaxError when the given code doesnt follow the proper syntax

```
>>> (12 / -3 >= -5) == true
(== (group (>= (/ 12 (- 3)) (- 5))) True)
>>>
```



You can see the clear similarities between the AST from the code and the AST I drew. The code does a preorder DFS on the tree, meaning it prints root, then left, then right. Following this in the true gives the same answer the program gave.

There are no failing tests, but I would like to expand my limitations for expressions with ++, --, and other shorthands for simplification.